Empirical Software Engineering (2026) 31:26
https://doi.org/10.1007/s10664-025-10762-7

Repository-level code completion with adaptive segmentation and fused retrieval

Yuanyuan Shen[1] · **Pinle Qin**[1] · **Kaiyi Zhao**[1] · **Fuwei Zhang**[1] · **Fan Zhang**[2] · **Guiji Li**[3]

Received: 8 April 2025 / Accepted: 3 November 2025 / Published online: 5 December 2025

Abstract
Code completion is of significant importance for improving development efficiency, reducing errors, and improving code quality. The development of code completion models is continuously evolving and has achieved significant progress, but existing code completion models still have limitations in accuracy for repository-level code-completion tasks that span multiple files and have a large number of cross-file dependencies. State-of-the-art retrieval augmented generation (RAG) results have motivated the use of the context of the entire codebase for repository-level code completion tasks to improve the prediction accuracy of large language models. In this study, we target RAG techniques to address the issues of constraints from current code completion models. To address the issue of code logic fragmentation caused by fixed-line segmentation, we prioritize code logical integrity and propose an adaptive segmentation strategy combined with a fused retrieval framework based on RAG technology and code structure analysis. We propose a structure-aware adaptive segmentation strategy that constructs functionally complete retrieval databases by analyzing structural units such as classes and functions. We use the fusion retrieval mechanism integrating both textual and functional similarity matching to enhance the quality and relevance of retrieved cross-file contexts. Experimental results on the CrossCodeEval benchmark demonstrate that our fused retrieval method achieves significant performance improvements over existing models.

Keywords Repository-level code-completion · RAG · Adaptive segmentation · Fused retrieval

Pinle Qin, Kaiyi Zhao, Fuwei Zhang, Fan Zhang, and Guiji Li contributed equally to this work.

Communicated by: Xin Xia.

Extended author information available on the last page of the article

1 Introduction

The earliest research on code completion dates back to SPELL (Kukich 1992) in 1971 for checking spelling errors in code. Early code completion was mainly based on the text autocomplete function, and the suggestions given by the early tools disregarded the syntax rules of the programming language and required many manual corrections (B et al. 2020). In recent years, machine learning and artificial intelligence techniques have been widely used for code completion. By training models to predict code structures and patterns, the system can provide more intelligent and personalized completion suggestions. In challenging repository-level code completion scenarios, the system needs to utilize cross-file contexts to complete variable names, lines, classes, or function calls within the current-file context. Due to a lack of understanding of the context and structure of the entire code repository, including repository-level user-defined application interfaces and inter-module dependencies, the performance of the code-completion models is limited. Recently, transformer (Vaswani et al. 2017)-based large language models (LLMs) such as the GPT series (OpenAI 2025; Brown et al. 2020) have reached the state-of-the-art in natural language processing tasks. Subsequently, LLMs tailored by researchers specifically for programming languages, such as StarCoder (Li et al. 2023), CodeLlama (Roziere et al. 2023), and DeepSeek-Coder (Guo et al. 2024) have shown robust performance in various tasks such as code-text summarization (Zhu et al. 2024) code repair (Jin et al. 2023), and generation (Chen et al. 2024b, a). The rapid development of LLMs has brought new vitality to the solution of code-completion tasks. Despite their encouraging performance and progress, LLMs rely on a large number of datasets, and this reliance limits their ability to perform on unseen data after training. Additionally, most of them correlate the code at the location to be completed with the current code context (Izadi et al. 2024) for code-completion tasks.

Integrating cross-file contexts is critical to enhancing the accuracy and relevance of automatic code-completion systems (Liang et al. 2024). To overcome this challenge, a promising solution is the adoption of Retrieval Augmented Generation (RAG) techniques, which supplement LLMs by retrieving external data in response to queries, ensuring more accurate and timely outputs (Huang and Huang 2024). For the repository-level code-completion task, recent approaches use the RAG technique to provide a new retrieval mechanism, i.e., fetching cross-file contexts within the same repository, thus providing augmented information to large language models (Zhang et al. 2023; Ding et al. 2023; Wu et al. 2024). Ding et al. (2024) proposed the CoCoMIC framework that incorporates cross-file contexts to jointly learn intra- and cross-file contexts by fine-tuning pre-trained code LLMs. Shrivastava et al. (2023) proposed the RepoFusion framework for training models to incorporate repository-level contexts. Liang et al. (2024) proposed the Repofuse framework which incorporates two types of contexts: analogical contexts and principal contexts, and proposed the rank-truncated generation technique to condense the prompt size and improve the inference efficiency. However, the works in Ding et al. (2024) and Liang et al. (2024) focused only on Python, while the work in Shrivastava et al. (2023) utilized the Fusion-in-Decoder (FiD) (Ding et al. 2021) method to integrate retrieved contexts but was limited to Java. Neither supports other programming languages.

To better evaluate the system's ability to retrieve the most relevant code chunks from cross-files, to predict the code to be completed using both cross-file and in-file contexts, and to handle the complex task that encompasses both of the above, Liu et al. (2023) and Ding

et al. (2023) proposed the RepoBench and CrossCodeEval benchmarks, respectively, where the former contains two programming languages and the latter contains four programming languages. Bouzenia et al. (2024) proposed an executable benchmark called DyPyBench, and introduced tools named SelfPiCo (Xue et al. 2024) and ExecutionAgent (Bouzenia and Pradel 2025) to assist researchers in performing tests more efficiently within executable projects. The authors provided effective benchmarks for later evaluating the utilization of context and code retrieval methods in code-completion research. For example, Wu et al. (2024) proposed a selective retrieval of context strategy to improve inference efficiency while enhancing accuracy. The authors demonstrated the superiority of their proposed method on the CrossCodeEval benchmark. However, these works truncate cross-file code to a fixed number of lines to construct a database of retrieved code chunks, which provides reference information for LLMs but ignores the structural nature of the code.

Figure 1 illustrates the difference between code chunks segmented at fixed lines and those segmented after considering the code structure. The original code before segmentation is obtained from CrossCodeEval (Ding et al. 2023). When segmenting code at fixed lines, as shown in Fig. 1(a), the functions of "begin_stream" and "stream" are split, compromising the logical integrity of the code. This results in the loss of partial parameters and structural features, which may lead the retriever to retrieve incomplete cross-file contexts and, consequently, cause the generator to produce incorrect results. Logical integrity in code entails the complete retention of function signatures/parameter lists and the full representation of control flow structures, aiding developers in more easily reading and understanding the code's functionality and logic. In contrast, Fig. 1(b) shows a code chunk segmented based on an unfixed-line code structure. We adopt this approach to preserve the logical integrity of the code, enabling the model to obtain clearer and higher-quality reference code information.

In this study, we aim to demonstrate the general applicability of the advanced retrieval method with code structure information to the repository-level code completion problem. We evaluate our approach using multiple LLMs on the CrossCodeEval. Comparative experiments against several baselines demonstrate that our approach achieves improvements in Exact Match (EM) scores, with performance gains of 10.7% for Python and 20.4% for Java, respectively. The experimental results validate the effectiveness of our approach across diverse LLMs and programming languages. To summarize, the main contributions of this study are as follows.

- We propose an adaptive segmentation strategy to construct a functionally complete retrieval database. Unlike fixed-line segmentation that leads to code information loss, we consider the effect of the logical integrity of the code on repository-level code completion and perform adaptive segmentation based on the size of the code structures such as classes, functions, and exception handling to improve the quality of reference code.
- We present a fused retrieval framework that includes textual similarity retrieval and functional similarity retrieval. Taking advantage of LLM generation, an LLM performs functional summarization of the current file and cross-file contexts for further functional matching, which is combined with textual matching to provide higher-quality prompts. The method fuses textual and functional similarity retrieval to guide the LLM in completing the inference of the code to be completed.
- In the comparative evaluation, the experimental results demonstrate the potential of adaptive segmentation and fused retrieval with RAG techniques for solving repository-level code completion. New solutions and tools for solving this task are presented.

```
# stop_conditions: List of strings or integer token IDs that will end the sequence
 # settings: ExLlamaAltGeneratorSettings
 # encode_special_characters: Set to true to tokenize "</s>" etc.
 def begin_stream(self, prompt: str, stop_conditions: list, max_new_tokens: int, gen_settings: Settings,
        encode_special_characters = False):
        assert isinstance(prompt, str), "ExLlamaAltGenerator does not support batched generation"
        # Tokenize prompt and limit length to allow prompt and (max) new tokens within max sequence length
        max_input_tokens = self.model.config.max_seq_len - max_new_tokens
        self.remaining_tokens = max_new_tokens
        input_ids = self.cached_tokenize(prompt, encode_special_characters)
        applied_input_ids = input_ids[:, -max_input_tokens:]
```

```
for ss in self.stop_strings:
        self.max_stop_tokens = max(self.max_stop_tokens, self.get_num_tokens(ss) + 2)
 self.settings = gen_settings
 # Start generation
 self.gen_begin_reuse(applied_input_ids, gen_settings)
 # Get the next chunk of text in the stream
 #
 # Returns stream_chunk: str, EOS: bool
 def stream(self):
    # Check total response length
```

(a) Fix-sized cross-file context.

```
def cached_tokenize(self, text: str, encode_special_characters = False):

    if text in self.tokenizer_cache:
        return self.tokenizer_cache[text]

    while len(self.tokenizer_cache) >= MAX_CACHED_STRINGS:
        # Always removes oldest entry, as of Python 3.7
        del self.tokenizer_cache[next(iter(self.tokenizer_cache))]

    new_enc = self.tokenizer.encode(text, encode_special_characters = encode_special_characters)
    self.tokenizer_cache[text] = new_enc
    return new_enc
```

```
# Generate and return a full prompt completion. See begin_stream() above

 def generate(self, prompt: str, stop_conditions: list, max_new_tokens: int, gen_settings: Settings,
    encode_special_characters =False):
    self.begin_stream(prompt, stop_conditions, max_new_tokens, gen_settings, encode_special_characters)
    response = ""
    while True:
        chunk, eos = self.stream()
        response += chunk
        if eos: break

    return response
```

(b) Structure-based cross-file context.

Fig. 1 Different forms of retrieval codes

The remainder of this study is structured as follows. Section 2 gives a summary of the related works to LLMs applied in the field of software engineering and repository-level code completion methods. Section 3 introduces the preliminaries knowledge and Section 4 displays the adaptive segmentation and fused retrieval method with RAG techniques architecture. In Section 5, the evaluation setup is presented. In Section 6, the evaluation results are provided with discussions. A review of threats to validity is given in Section 7. Finally, a conclusion of this study is drawn in Section 8.

2 Related Work

2.1 Code LLMs

In recent years, a large number of LLMs tailored for code-related tasks aimed at code comprehension and generation have been widely used in automation tasks in software engineering domains, including code completion, to improve development efficiency. A series of pre-trained models, from closed-source models at the beginning to open-source models later on, have been applied to code completion scenarios such as Codex (Chen et al. 2021), PaLM-Coder (Chowdhery et al. 2023), the CodeGen family (Nijkamp et al. 2023b), CodeT5 (Wang et al. 2021), CodeT5+ (Wang et al. 2023), StarCoder (Li et al. 2023), CodeGeeX (Zheng et al. 2023), CodeFuse (Di et al. 2024), PangguCoder (Christopoulou et al. 2022), CodeLlama (Roziere et al. 2023), and SantaCoder (Allal et al. 2023). Several benchmarks have been successively proposed for assessing the performance of these code LLMs by given the current-file contexts, such as HumanEval (Chen et al. 2021), APPS (Hendrycks et al. 2021), MultiPL-E (Athiwaratkun et al. 2023), MBXP, Multilingual HumanEval, and MathQA-X (Phan et al. 2024). The code completion ability of the LLMs on these benchmarks proved to be effective. The LLMs are effective in predicting whole lines of code (Izadi et al. 2022) (Biderman et al. 2023), but there are still limitations in using them for repository-level code completion tasks. Due to the limitation on the number of tokens, LLMs cannot fully understand the extensive context of the entire codebase (Phan et al. 2024). These models are primarily trained on open-sourced public codebase; most of them focus primarily on local information within a single-file, ignoring cross-file contextual dependencies specific to individual repositories, and lack comprehensive support for entire repositories (Liang et al. 2024).

2.2 Retrieval Augmented Generation

Generative AIs powered by LLMs have achieved impressive results, but they face some challenges such as the inability to access or utilize up-to-date information or specific knowledge outside of the training data (Mallen et al. 2023) and the risk of data leakage (Carlini et al. 2021). RAG techniques address the above challenges by introducing a retrieval component. This retrieval component can retrieve relevant information from external knowledge bases or memories before generating a response. In this way, the generator of the model can utilize this retrieved information to generate more accurate and informative responses. The specific paradigm is as follows: given an input query, the retriever looks for matching data sources and then interacts with the generator using the retrieved results to enhance the

generation process (Zhao et al. 2024). This technique has been applied to applications in different domains such as image captioning (Ramos et al. 2023), video captioning (Chen et al. 2023), QA systems (Kim et al. 2024) and code summarization (Li et al. 2021). When applying RAG techniques to code-related tasks, it is crucial to effectively combine code retrieval and generation. The former can use Abstract Syntax Trees (ASTs) or dense representations (Wang et al. 2021), as well as code edit distances (Li et al. 2021) to identify similar code chunks. The latter can use generative models to generate code or natural language (Tsai et al. 2024; Liu et al. 2024a).

2.3 Repository-level Code Completion

Repository-level code completion is an actively researched direction in the field of automated code completion, which aims to provide more intelligent and context-aware code completion suggestions. Compared with traditional code completion tools, repository-level code completion considers the context of the entire code repository, not just the currently edited file or function. Recent research works based on LLMs to better utilize the knowledge provided in in-file and cross-file contexts have made significant progress in repository-level code auto-completion. For example, Liao et al. (2024) proposed the A$\hat{3}$-CodGen framework to mine and utilize local context information, third-party libraries, and global modules. Eghbali and Pradel (2024) proposed an iterative retrieval and code generation approach that involves using the results generated by the LLM in the previous iteration for retrieving relevant API reference information, and then connecting the retrieved results with the previous context for the next round of retrieval and generation. Wang et al. (2025b) proposed a reinforcement learning framework named RLCoder, where the retriever is able to iteratively learn by obtaining feedback from evaluators. Wang et al. (2025a) and Huang et al. (2024) used agents to accomplish automated code completion, including invoking multiple tools or components to collaborate with each other.

LLM-augmented methods with RAG techniques such as RepoFuse (Liang et al. 2024), RepoFormer (Wu et al. 2024) and the works in Ding et al. (2024) and Shrivastava et al. (2023) achieve improved performance in repository-level code completion tasks. However, current repository-level code completion methods still have some limitations in exploring the inference performance of models that rely on retrieving external knowledge. Firstly, many learning-based models lack generalizability. For instance, the Repofuse framework only supports Python, while RLCoder supports only Python and Java. Secondly, previous work (Ding et al. 2023; Wu et al. 2024; Liu et al. 2023) mostly adopted fixed-line truncated code blocks, which can split the complete code structure. This may cause the retriever to miss some parameters during retrieval. Additionally, this partial information could confuse the generator, leading to incorrect function calls. In the following sections, we use adaptive segmentation and fused retrieval methods to solve the repository-level code-completion problem for multiple programming languages with high accuracy and efficiency.

3 Preliminaries

In this section, we provide an initial introduction to the concept of repository-level code completion tasks and the RAG technique.

Problem Description A real-world repository-level code completion problem can be defined as completing the subsequent code snippets at the cursor position of an unfinished code file in a private repository. We formalize this problem as a triple (CC_{uur},*Y*,*F*), where CC_{uur} means the current unfinished file, *Y* represents the ground-truth code lines that need to be completed, and *F* is a list of other files in the repository.

Retrieval Augmented Generation We follow the description in Ding et al. (2023) to execute RAG for repository-level code completion in the following stages:

- Indexing: all files in *F* are split into code chunks of different sizes based on their code structures to build a retrieval repository.
- Query and Retrieval: The current file CC_{uur} is used as a query to retrieve *K* similar code chunks $cs_1, cs_2, \ldots, cs_K$ to form the cross-file context CC_{ros}, where the size of *K* is not fixed. Due to the different sizes of code chunks, and to avoid a decrease in model inference efficiency caused by too long code chunks, we determine the number of *K* according to the number of tokens received by the model.
- Generation: CC_{uur} and CC_{ros} are concatenated into a prompt to guide the model in generating the code $\hat{Y}$ to be complemented.

4 Approach

Figure 2 illustrates the overview of our code completion framework, which consists of two main components. First, an adaptive segmentation strategy divides the source code into different chunks to build the retrieval database. Second, an LLM-augmented module generates functional summaries of code chunks, retrieves relevant ones via text and functional similarity, performs re-ranking, and combines similar retrieval contexts with the current file context to form a prompt. This prompt is then used by the LLM to generate the final code that needs to be completed. In the following, we detail each component of the fused retrieval framework.

4.1 Adaptive Segmentation Strategy

Clear code structure is the key for programmers to organize their code files effectively, especially when multiple programmers are working together in the same repository; it can significantly reduce communication costs and improve the efficiency of collaboration. In code review, good code structure enables reviewers to quickly locate code chunks, making it easier to identify potential problems. In addition, a good code structure promotes code reuse and greatly improves development efficiency by encapsulating reusable code into libraries or modules. Therefore, code structure is of decisive importance for programmers to write and manage code repositories. However, existing retrieval-based code completion methods often overlook the semantic granularity provided by code structure. For instance, fixed-line segmenting (Ding et al. 2023) may split semantically cohesive units, such as method bodies or control blocks, leading to incomplete context during retrieval. The split-aggregate candidate code snippet construction strategy (Wang et al. 2025b) relies on blank lines to divide and aggregate code snippets. This approach may fail to preserve syntactic boundaries, espe-

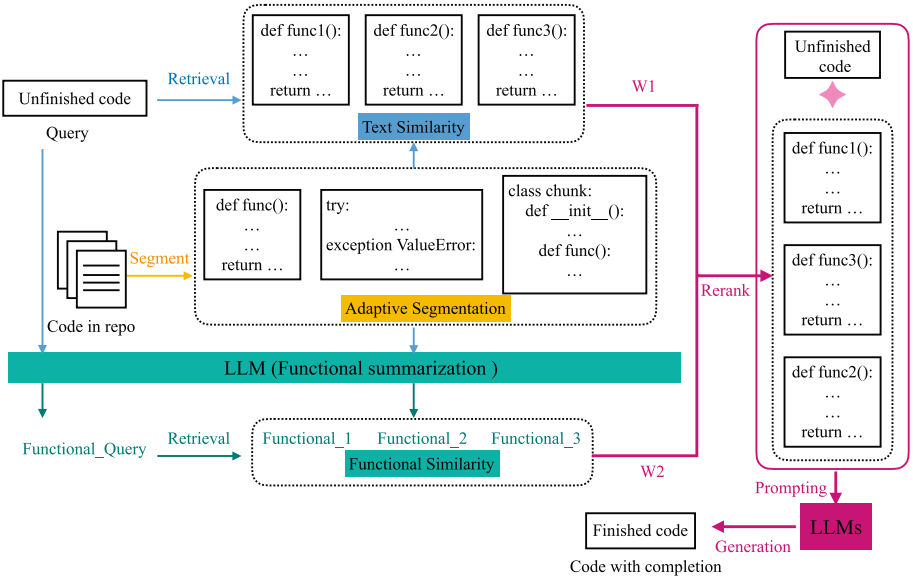

Fig. 2 Our proposed fused retrieval architecture for repository-level code completion

cially when code formatting is inconsistent. It can inadvertently merge distinct logical units or split meaningful blocks due to its lack of syntax awareness, which weakens the quality and relevance of retrieved candidates.

To address this issue, we argue that an effective segmenting strategy should ensure comprehensive code coverage while respecting underlying syntactic structure boundaries. To this end, we propose a structure-aware adaptive segmentation strategy. This approach builds upon the sliding window mechanism to guarantee full code coverage, while incorporating syntactic analysis to dynamically adjust window boundaries and preserve the integrity of fundamental semantic units. Unlike prior work that extracts fixed-line code snippets, our method balances coverage and semantic completeness by integrating static syntactic information into the sliding mechanism. The number of lines of code chunks is not fixed in our retrieval database. We construct the retrieval database according to the following rules and the algorithm 1:

1. **Sliding window as the triggering mechanism:** We propose to construct the retrieval database based on basic blocks of code. We define code chunks that cannot be segmented, such as Function, Class, If, For, While, Try, With, etc. These structural snippets are called basic blocks. All source code are segmented with a sliding window, and we set the sliding window size to N and the sliding size to M. There will be a situation where two blocks overlap by $N - M$ lines. This mechanism ensures continuous coverage even in regions lacking explicit structure, thereby avoiding the omission of critical context.
2. **Syntax-based dynamic boundary adjustment:** When a segmentation point occurs within a basic block, it is relocated to the start of the block to preserve structural integrity. When sliding backward, the start position of the segmentation cannot fall within

a basic block; if it does, the starting position is moved a few lines forward to ensure the integrity of the basic block. We prioritize extracting the innermost basic block at the current segmentation position. If the current position lies outside any basic block, a sliding window is used to extract code. If the window ends within a basic block, it is aligned to the basic block's end to preserve the integrity of logical units such as functions.

This design provides several key advantages. It ensures that each chunk is adjusted to form a functionally coherent unit. The approach also maintains contextual coherence through the combined use of overlapping sliding windows and syntax-driven boundary alignment, which preserves local context and minimizes the risk of losing critical surrounding information. Furthermore, the strategy demonstrates strong adaptability. It naturally produces finer-grained, more focused chunks that preserve structural integrity.

When we set a window size of 5 and a slide size of 3, the current nested block shown in Fig. 3 is processed as follows: when $current\_pos = 54$, the sliding window starts at line 54, and after structural alignment, the system extracts a block from lines 55 to 56, covering the core assignment within the __init__ method; when $current\_pos = 57$, which is not inside any function block, the window initially spans lines 57 to 61. Since it ends at the start of the bark2 function, it is extended to the end of that function (line 62), resulting in a chunk from lines 57 to 62 that includes both bark1 and bark2. When $current\_pos = 60$, the window spans lines 60 to 64. As it ends at the beginning of the bark3 function, it is aligned to line 65, which is the end of bark3, yielding a chunk from lines 60 to 65 that contains both bark2 and bark3. Finally, when $current\_pos = 63$, the window covers lines 63 to 65. Since this range already falls entirely within the bark3 function and reaches its end, no further alignment is needed, and the block from lines 63 to 65 is extracted, containing only bark3. In total, four valid code chunks are generated.

It is worth noting that, under our strategy, the bark3 function is extracted as an isolated code block, whereas bark1 and bark2 appear together in the same chunk. This difference may seem asymmetric at first, but it does not stem from semantic differences among the functions. Instead, it arises from the interplay of code layout spacing, end-boundary structural alignment, and sliding window dynamics. Since our strategy only adjusts the start position backward when it lies within a code block and ensures completeness by aligning the end boundary, adjacent functions separated by a single blank line are likely to be co-covered by overlapping windows. In contrast, the final function (bark3) has no subsequent function, making its standalone extraction inevitable under certain starting positions. Thus,

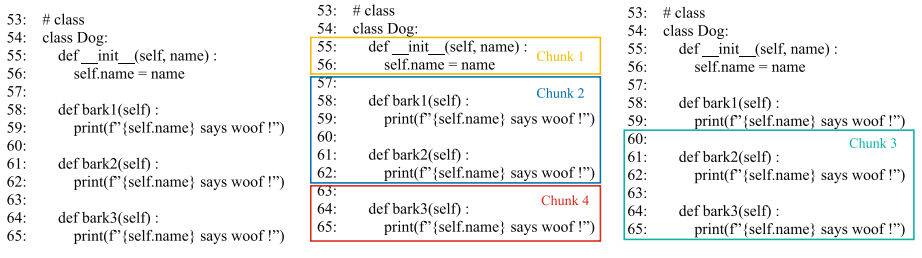

Fig. 3 The adaptive segmentation strategy

the observed variation results from the consistent application of the same rule across all contexts, rather than any semantic bias.

Algorithm 1 The proposed adaptive segmentation algorithm.

Input: Source code file f, window size N, slide size M
Output: List of code chunks CS

$CS \leftarrow \emptyset$;
$basic\_blocks \leftarrow$ extract all basic blocks from f using AST parsing or syntax analysis;
$block\_list \leftarrow$ a list of $[start\_line, end\_line]$ for each basic block;
$current\_pos \leftarrow 1$;
$saved\_blocks \leftarrow \emptyset$;
while $current\_pos \leq$ *total lines in* f **do**
  $inside\_block \leftarrow$ find the innermost basic block where $current\_pos \in [start, end)$;
  if $inside\_block \neq None$ **then**
    $[s, e] \leftarrow [start\_line, end\_line]$ of $inside\_block$;
    if $[s, e] \notin saved\_blocks$ **then**
      $CS \leftarrow$ extract lines s to e and add to CS;
      $saved\_blocks \leftarrow [s, e]$;
    end
    $current\_pos \leftarrow e + 1$;
  else
    $window\_start \leftarrow current\_pos$;
    $window\_end \leftarrow \min(current\_pos + N - 1,$ total lines in $f)$;
    $inside\_block \leftarrow$ find the innermost block containing $window\_end$;
    if $inside\_block \neq None$ **then**
      $[s, e] \leftarrow [start\_line, end\_line]$ of $inside\_block$;
      $window\_end \leftarrow e$;
    else
      $window\_end \leftarrow window\_end$;
    end
    if $[window\_start, window\_end] \notin saved\_blocks$ **then**
      $CS \leftarrow$ extract lines $window\_start$ to $window\_end$ and add to CS;
      $saved\_blocks \leftarrow [window\_start, window\_end]$;
    end
    $current\_pos \leftarrow current\_pos + M$;
  end
end
return CS

After code segmentation, we examined the length and type distribution of the code blocks. For the Python language, all files were split into 84,001 code blocks, totaling 2,154,845 lines, with the largest block containing 871 lines. Function structures, class structures, and for-loop structures were the most common, with 37,075 blocks, 16,997 blocks, and 11,853 blocks, respectively. For the Java language, all files were split into 85,618 code blocks, totaling 1,146,011 lines, with the largest block containing 776 lines. If-statement structures, method structures, and class structures were the most common, with 20,189 blocks, 26,337 blocks, and 5,175 blocks, respectively. For the C# language, all files were split into 39,907 code blocks, totaling 585,962 lines, with the largest block containing 586 lines. If-statement structures, public method structures, and for-loop structures were the most common, with 12,407 blocks, 4,190 blocks, and 2,179 blocks, respectively. For the TypeScript language, all files were split into 71,628 code blocks, totaling 1,038,512 lines, with the largest block containing 661 lines. If-statement structures, function structures, and interface structures

were the most common, with 11,766 blocks, 4,548 blocks, and 2,405 blocks, respectively. We do not perform intra-block truncation on these longer blocks but instead control them uniformly according to the cross-file context truncation size. For example, if the token size of the top reference block is 5000, when the cross-file context truncation size is 512, the first 512 tokens of this block are retained; and when the truncation size is 4096, the first 4096 are retained. As the truncation size expands, a larger portion of the original code block is preserved, thereby reducing the risk of losing critical contextual information.

Based on these rules and the algorithm 1, we split the source code F in the dataset, excluding the code to be completed, that is unfinished code, into chunks of unfixed-line code. We construct a retrieval database, *CS*, where each chunk is denoted as cs_i. Additionally, each chunk is named to include information about its original file, allowing us to distinguish whether it originates from the same repository or not. Subsequently, as shown in Fig. 2, the unfinished code is used as a query to retrieve blocks from the same repository in *CS* that are similar to the query text using the retrieval algorithm. Next, the unfinished code and all blocks cs_i are fed into an LLM to summarize their intent, and then blocks that are functionally similar to the query are retrieved.

4.2 LLM-augment Module

In this section, we first introduce how to conduct retrieval based on a query, and then utilize the retrieved results to guide the generation process of large models.

Textual Retrieval In this step, we integrate both textual and functional similarity retrieval. When calculating textual similarity, we observe the excellent performance of the BM25 algorithm (Robertson et al. 2009), as noted in papers Ding et al. (2023) and Liang et al. (2024). We employ BM25 to compute the similarity $textual\_sim_{CS_i}$ between the query CC_{uur} and the cross-file snippets cs_i from the same repository in the retrieved database. The definition is as follows:

$$textual\_sim_{CS_i} = BM25(CC_{uur}, cs_i), i \in L. \tag{1}$$

Here, CC_{uur} represents the query, and cs_i belongs to the set *CS*, which denotes the basic blocks or other code chunks in the retrieval database. The L indicates the total number of cross-file code chunks in the database.

Functional Retrieval Subsequently, we look up code chunks that are functionally similar based on functional similarity. Since textually similar code is not always functionally similar (Wu and Cao 2024), we match the functionality of the code during retrieval. LLMs excel in code-related tasks, easily grasping the semantics and logic of source code. Therefore, we propose combining prompts with LLMs to ensure logical integrity in source code summarization. We select a large language model named DeepSeek-v2 for summarizing source code because it is renowned for its performance in knowledge, mathematics, reasoning, and programming tasks. Additionally, the DeepSeek-v2 model supports contexts of up to 32000 tokens and offers cost-effectiveness. Specifically, we first construct a prompt: *Prompt = "You are a source code summarization expert responsible for summarizing the intent and functionality of the source code I enter."*.

Second, we use DeepSeek-v2 to summarize the functional descriptions of query $f_{CC_{uur}}$ and cross-file code chunks f_{cs_i} while ensuring that the length of the source code input to the DeepSeek-v2 does not exceed the maximum context length of the model, which results in:

$$f_{CC_{uur}} = DeepSeek(Prompt, CC_{uur}), \tag{2}$$

$$f_{CS_i} = DeepSeek(Prompt, CS_i), \tag{3}$$

Next, we calculate the functional similarity $functional\_sim_{CS_i}$. The UniXcoder is a powerful pre-trained model for code representation and performs well on code summarization tasks (Guo et al. 2022). We use UniXcoder to obtain vector representations of functional descriptions and then use cosine similarity (Ding et al. 2023) to compute the functional similarity between vector representations as follows:

$$V_{CC_{uur}} = UniXcoder(f_{CC_{uur}}), \tag{4}$$

$$V_{CS_i} = UniXcoder(f_{CS_i}), \tag{5}$$

$$functional\_sim_{CS_i} = cosine\_similarity(V_{CC_{uur}}, V_{CS_i}), \tag{6}$$

Here, $V_{CC_{uur}}$ represents the vector representation of $f_{CC_{uur}}$, and V_{CS_i} represents the vector representation of f_{CS_i}.

Finally, we propose combining textual similarity and functional similarity as the final similarity score for query and cross-file code chunks:

$$sim_{CS_i} = \omega_1 functional\_sim_{CS_i} + \omega_2 textual\_simCS_i, \tag{7}$$

wherein ω_1 and ω_2 are the weights of functional similarity and textual similarity respectively. By reranking all similarity scores, we obtain a ranked list of code chunks that are textually and functionally similar:

$$cs_1, cs_2, \ldots, cs_K = Rerank(sim_{CS_1}, sim_{CS_2}, \ldots, sim_{CS_L}). \tag{8}$$

LLM Inference In this step, an LLM generates the code to be completed based on the current file context and the cross-file code chunks. We construct the prompt by concatenating the last *n* lines of the current file with up to *K* cross-file contexts. The resulting input is then truncated according to the LLM's token limit to maximize context utilization, instead of enforcing a fixed upper bound on *K*.

5 Experimental Setups

This section describes the details of the implementation. First, we describe the benchmark used for our experiments. Then, we introduce the experimental setting and metrics.

5.1 Benchmark

We evaluate the performance of our method using the state-of-the-art CrossCodeEval benchmark (Ding et al. 2023). CrossCodeEval is a robust dataset designed for assessing repository-level code completion methods. It includes code chunks in Python, Java, TypeScript, and C#, emphasizing the necessity of understanding context across different files for precise code predictions. This dataset simulates real-world programming tasks in various languages, demanding a sophisticated grasp of software repository interdependencies. Our study utilizes the four programming language subsets of CrossCodeEval to rigorously assess and validate our method's capability to produce accurate and context-aware code completions. As shown in Table 1, the Python subset contains 471 repositories with a total of 1368 files, and the number of examples, that is, code to be completed, is 2665. The Java subset contains 239 repositories with 745 files, and the number of code samples to be completed is 2139. The Typescript subset contains 193 repositories with 779 files and 3356 numbers of code to be completed. The C# subset contains 99 repositories with 642 files and 1768 numbers of code to be completed.

5.2 Configuration

We set the sliding window size to 10 and the sliding step size to 8 to perform the adaptive chunking strategy. We perform text similarity retrieval using the BM25 algorithm, functional summarization using DeepSeek-v2, and functional similarity retrieval using the UniXcoder model and cosine similarity function, where the UniXcoder is used to generate embeddings of code with the default embedding dimension of 768 for each token. When using BM25, we set the query used for retrieval to the last 5 lines of the in-file context. The LLMs we chose to perform inference are StarCoderBase with 1B, 3B, and 7B sizes (Li et al. 2023), StarCoder2 with 3B and 7B sizes (Lozhkov et al. 2024), and DeepSeek-Coder with 1B and 7B sizes (Guo et al. 2024). We use the performance of StarCoderBase-1B as the baseline. StarCoderBase represents a suite of code models trained on an extensive corpus encompassing over 80 programming languages, which is complemented by a substantial context window capable of processing 8192 tokens. StarCoder2 constitutes a family of advanced code generation models, distinguished by their training on a comprehensive set of 600+ programming languages, supporting context windows of size 16,384 tokens and sliding window attention of 4,096 tokens. DeepSeek Coder comprises a series of code language models, each of which undergoes pre-training on a project-level code corpus. These models all support 8k input tokens. We evaluate the truncation size of context varying from 512 to 4096.

We conduct all experiments on a 13th Gen Intel (R) Core (TM) i9-13900KF × 32 CPU machine equipped with an NVIDIA RTX A6000/PCIe/SSE2 GPU.

Table 1 Statistics of CrossCodeEval (Ding et al. 2023)

Feature	Python	Java	Typescript	C#
Repositories	471	239	193	99
Files	1368	745	779	642
Examples	2665	2139	3356	1768

5.3 Evaluation Metrics

We use Code Match and Identifier Match [44] as evaluation criteria, adhering to the definitions outlined in CrossCodeEval (Ding et al. 2023).

Code Match The Code Match metric serves as a direct comparative assessment between the inference code and the reference code, utilizing Exact Match (EM) and Edit Similarity (ES) as its primary measures. These metrics are instrumental in evaluating the precision of the code completion process, encompassing aspects such as identifiers, keywords, operators, delimiters, and literals.

Identifier Match The Identifier Match metric specifically evaluates the model's proficiency in accurately predicting APIs. This assessment involves parsing both the inferred code and the reference code to extract identifiers, thereby yielding two ordered lists of identifiers. Subsequently, a comparison is made between the inference identifiers and the reference identifiers, utilizing EM and F1 scores as its primary measures.

6 Experimental Results

In this section, we evaluate the proposed LLM-augmented retrieval method for the code completion task at the repository-level. We present the experimental results, and analyze the effectiveness of our method. The experimental results are presented in tables, with the best results in Tables 2, 3, 4, 5, 6, 8, 9, and 10 highlighted in bold.

6.1 Performance of Our Approach

To evaluate the effectiveness of our approach, we compared it with RLCoder (Wang et al. 2025b), RawRAG (Parvez et al. 2021), and RepoCoder (Zhang et al. 2023), using four large language models: StarCoderBase-7B (Li et al. 2023), StarCoder2-7B (Lozhkov et al. 2024), DeepSeekCoder-1B, and DeepSeekCoder-7B (Guo et al. 2024). We conducted performance evaluations on the Python and Java datasets of the CrossCodeEval benchmark, using Exact Match (EM) and Edit Similarity (ES) as evaluation metrics. The performance results for RLCoder, RawRAG, and RepoCoder were sourced from reference Wang et al. (2025b). According to Wang et al. (2025b), not all retrieved candidate fragments are useful. To address this issue, the authors proposed a stop-signal mechanism to evaluate the effectiveness of each candidate fragment, potentially eliminating the need for cross-file context. In the implementations of RLCoder, RawRAG, and Repocoder, if cross-file context is unnecessary, the truncation size for cross-file context is set to 0; otherwise, it defaults to 1536, with a retrieval query size of 512. In our approach, the query consists of the last five lines of the code to be completed, resulting in a variable token count. We set the cross-file context truncation size to 4096. We also validate our approach on the RepoEval (Liu et al. 2024b), which was created by Liu et al. and is an update version of the original RepoEval dataset (Zhang et al. 2023). We report the best results from the literature (Liu et al. 2024b), where GraphCoder's (Liu et al. 2024b) performance on the line-level tasks in Python and on the

Table 2 Code matching performances of different methods

CrossCodeEval	Code matching				
Model	Method	Python		Java	
		EM	ES	EM	ES
StarCoderBase-7B	RawRAG	22.33	69.60	22.16	67.80
	RepoCoder	23.15	70.71	22.53	68.22
	RLCoder	25.82	72.11	24.73	**69.08**
	Our	**30.21**	**73.48**	**27.86**	67.94
StarCoder2-7B	RawRAG	22.89	70.66	23.42	69.13
	RepoCoder	24.35	71.71	23.75	69.59
	RLCoder	27.17	73.24	26.23	70.51
	Our	**30.09**	**74.07**	**29.83**	**70.77**
DeepSeek-1B	RawRAG	19.74	67.68	18.89	62.47
	RepoCoder	20.23	68.78	19.59	62.35
	RLCoder	23.98	70.44	20.80	63.39
	Our	**28.11**	**71.94**	**25.29**	**65.77**
DeepSeek-7B	RawRAG	23.30	70.84	22.49	66.78
	RepoCoder	26.98	72.96	24.96	66.52
	RLCoder	30.28	74.42	26.09	67.31
	RepoCoder w/ RLCoder	30.32	74.79	26.98	67.81
	Our	**33.51**	**75.72**	**31.42**	**69.43**
RepoEval	Code matching				
Line-level	GraphCoder	46.6	69.42	**50.6**	**78.94**
	Our	**48.9**	**69.88**	48.75	74.42
API-level	GraphCoder	48.75	69.97	61.57	**82.66**
	Our	**52.45**	**72.74**	**62.1**	81.28

API-level and line-level tasks in Java was obtained using GPT-3.5-Turbo-Instruct[1], while the result on the API-level task in Python was obtained using CodeGen2-16B (Nijkamp et al. 2023a). The prompt, which is composed of the in-file query and the cross-file context, is limited to a length of 4096 when using GPT3.5-Turbo-Instruct and 2048 when using CodeGen2-16B. Our performance results on Python and Java are obtained using DeepSeek-Coder-7B, where the number of tokens in the in-file query is not fixed, and the number of tokens in the cross-file context is set to 4096.

From the experimental results shown in Table 2, it can be observed that the proposed method demonstrates effectiveness across all four models. Among all models, our approach achieves the best performance when using DeepSeekCoder-7B, with an EM score of 33.51, improving over RLCoder with DeepSeekCoder-7B by 10.7% on CrossCodeEval Python and by 20.4% on CrossCodeEval Java. Additionally, it outperforms RepoCoder enhanced with RLCoder by 10.5% on CrossCodeEval Python and by 16.5% on CrossCodeEval Java. Compared to GraphCoder, our approach performs well on Python, achieving an average improvement of 2.31 across all four evaluation metrics. On Java, one metric shows a slight improvement, while three metrics show a decline, resulting in an average decrease of 1.81. In the Java subset of the RepoEval dataset, method density is high, and the highly modular design causes code logic to be distributed across multiple methods and classes. The code context graph (CCG) proposed in Liu et al. (2024b) integrates control flow, data dependencies, and control dependencies into a multi-level semantic graph. By leveraging program

[1] https://platform.openai.com/docs/models/gpt-3.5-turbo

Table 3 Code matching performance with and without functional similarity retrieval over all LLMs

Truncation 4096	Code matching								
Model	Method	Python		Java		Typescript		C#	
		EM	ES	EM	ES	EM	ES	EM	ES
StarCoderBase-1B	w/o FSR	18.35	67.13	15.61	64.04	9.42	46.42	18.21	68.53
	w/o TSR	21.99	68.17	19.40	63.88	8.37	45.12	19.17	66.65
	Our	**22.66**	**68.66**	**19.50**	**64.33**	**10.25**	**46.71**	**20.36**	**68.68**
StarCoderBase-3B	w/o FSR	22.25	68.79	19.87	64.85	10.91	47.41	22.00	69.20
	w/o TSR	25.56	69.46	23.60	66.42	11.17	47.49	22.57	69.29
	Our	**27.28**	**71.18**	**25.57**	**67.53**	**12.13**	**48.04**	**23.93**	**70.08**
StarCoderBase-7B	w/o FSR	24.92	70.5	22.25	65.75	12.16	47.89	23.81	67.97
	w/o TSR	29.38	71.84	26.71	67.93	12.66	48.52	24.38	69.04
	Our	**30.21**	**73.48**	**27.86**	**67.94**	**13.68**	**49.06**	**26.13**	**69.61**
StarCoder2-3B	w/o FSR	25.48	71.08	22.11	65.86	11.35	43.20	24.55	70.29
	w/o TSR	28.98	71.86	25.55	66.94	12.60	43.86	25.17	70.51
	Our	**29.64**	**73.63**	**27.68**	**68.49**	**12.87**	**44.01**	**26.41**	**70.95**
StarCoder2-7B	w/o FSR	25.70	71.69	23.98	67.70	13.11	47.18	26.02	71.89
	w/o TSR	29.38	72.32	27.86	69.17	13.62	49.91	26.41	71.93
	Our	**30.09**	**74.07**	**29.83**	**70.77**	**14.45**	**49.74**	**28.17**	**73.30**
DeepSeek-1B	w/o FSR	23.30	69.26	18.37	62.30	16.90	62.89	21.21	67.42
	w/o TSR	26.84	70.27	23.46	64.28	17.94	63.50	21.83	68.07
	Our	**28.11**	**71.94**	**25.29**	**65.77**	**18.86**	**64.31**	**23.13**	**68.91**
DeepSeek-7B	w/o FSR	28.37	73.24	24.92	66.43	22.11	66.95	24.49	65.91
	w/o TSR	32.80	74.52	29.67	68.71	23.60	68.16	24.83	67.07
	Our	**33.51**	**75.72**	**31.42**	**69.43**	**23.93**	**68.18**	**26.98**	**68.40**

slicing techniques, the CCG can trace backward from the completion point to capture all variables and control conditions affecting its execution, effectively preserving critical long-range semantic information, and thus demonstrating superior performance in Java scenarios. In contrast, our adaptive segmentation strategy, although maintaining syntactic integrity, remains confined to local code fragments and struggles to model multidimensional semantic relationships such as cross-line and cross-method collaborations. This results in retrieved contexts lacking completeness and failing to adequately reflect long-range program dependencies, indicating limitations in adapting to the global class structure inherent in Java.

We evaluated the model's performance when further increasing the truncation size. Since the maximum token context length of the StarCoderBase model is 8192, setting the cross-file context truncation size to 8192 results in the error message: "Input prompt is too long and exceeds limits of 8192." Therefore, we evaluated the performance of our approach with the StarCoderBase-1B, StarCoderBase-7B, StarCoder2-7B, DeepSeekCoder-1B, and DeepSeekCoder-7B models when the truncation size was set to 7500. On Python, the EM and ES scores for the five models were: 23.71 and 68.89, 29.72 and 73.62, 31.03 and 73.83, 29.04 and 72.64, and 34.71 and 76.81. On Java, the EM and ES scores were: 19.78 and 62.97, 27.44 and 66.55, 26.98 and 66.85, 23.89 and 63.82, and 29.45 and 67.0. Compared with the model's performance when the truncation size was set to 4096 (as shown in Tables 2 and 3), we observed that some metrics improved while others declined. A truncation size of 4096 appears to be the optimal configuration.

Table 4 ID matching performance with and without functional similarity retrival over all LLMs

Truncation 4096	ID matching								
Model	Method	Python		Java		Typescript		C#	
		EM	F1	EM	F1	EM	F1	EM	F1
StarCoderBase-1B	w/o FSR	27.58	57.02	24.31	54.08	14.87	38.62	22.4	47.88
	w/o TSR	32.05	60.69	27.54	**56.46**	13.44	37.22	23.47	48.01
	Our	**33.1**	**61.17**	**27.72**	56.37	**15.46**	**39.42**	**24.66**	**50.26**
StarCoderBase-3B	w/o FSR	32.42	61.39	29.08	57.21	16.03	39.31	26.30	52.06
	w/o TSR	35.82	62.80	32.63	59.56	16.42	39.18	27.09	52.27
	Our	**37.82**	**64.66**	**34.69**	**60.94**	**17.49**	**40.31**	**27.94**	**53.24**
StarCoderBase-7B	w/o FSR	34.3	62.83	31.65	58.74	17.49	39.89	28.00	52.01
	w/o TSR	39.12	65.58	36.19	61.86	17.76	40.33	29.41	52.98
	Our	**40.26**	**66.86**	**37.49**	**62.11**	**19.61**	**41.69**	**29.86**	**54.31**
StarCoder2-3B	w/o FSR	35.72	63.63	31.14	58.59	16.54	37.87	28.9	53.72
	w/o TSR	39.01	66.07	34.90	60.89	**18.00**	38.87	29.81	54.71
	Our	**40.11**	**66.97**	**37.26**	**62.42**	17.97	**39.17**	**30.77**	**55.00**
StarCoder2-7B	w/o FSR	35.61	64.03	33.05	60.76	18.65	40.99	30.66	55.57
	w/o TSR	39.68	66.12	37.21	63.04	19.40	43.36	30.94	56.07
	Our	**40.71**	**67.40**	**39.18**	**64.88**	**19.99**	**43.74**	**32.75**	**57.47**
DeepSeek-1B	w/o FSR	32.98	61.72	26.98	54.92	23.48	55.57	26.07	51.33
	w/o TSR	37.18	63.84	32.21	58.02	24.61	56.34	26.47	52.14
	Our	**38.20**	**65.68**	**34.08**	**59.53**	**25.86**	**57.44**	**27.88**	**52.78**
DeepSeek-7B	w/o FSR	38.72	66.23	34.22	60.30	29.11	60.76	29.30	51.89
	w/o TSR	43.87	68.59	39.11	63.72	30.33	62.25	29.81	53.46
	Our	**45.03**	**70.47**	**40.95**	**64.38**	**31.26**	**62.49**	**31.79**	**54.98**

Table 5 Code matching performance of different segmentation strategies at varies truncation sizes

Model StarCoderBase-1B	Code matching										
Truncation size	Segmentation strategy	Python		Java		Typescript		C#		AVG	
		EM	ES	EM	ES	EM	ES	EM	ES	EM	ES
512	Fixed-size	5.82	59.31	4.11	59.70	**3.34**	**45.56**	7.69	62.90	5.24	**56.87**
	Adaptative	**6.00**	**60.06**	**5.38**	**60.09**	3.10	43.06	**8.37**	**62.93**	**5.71**	56.54
1024	Fixed-size	8.86	59.49	6.12	60.30	4.32	**45.70**	10.24	63.73	7.39	57.31
	Adaptative	**9.94**	**60.73**	**8.65**	**60.99**	**4.86**	44.83	**11.48**	**64.27**	**8.73**	**57.71**
2048	Fixed-size	12.72	63.84	9.26	61.30	6.59	**46.97**	13.52	65.53	10.52	59.41
	Adaptative	**14.33**	**65.12**	**11.97**	**62.01**	**7.24**	45.60	**15.84**	**66.33**	**12.35**	**59.77**
4096	Fixed-size	16.14	65.92	13.00	62.86	8.55	**47.46**	16.57	68.02	13.57	61.07
	Adaptative	**18.35**	**67.13**	**15.61**	**64.04**	**9.42**	46.42	**18.21**	**68.53**	**15.4**	**61.53**
7500	Fixed-size	19.21	66.29	16.69	**60.89**	8.46	**44.65**	18.72	66.47	15.77	**59.58**
	Adaptative	**20.34**	**67.3**	**16.97**	60.83	**8.46**	43.16	**19.74**	**66.54**	**16.38**	59.46

We further analyzed the efficiency of the proposed method. Taking Python as an example, the time for chunking using a segmentation strategy is 30 seconds. The time for summarizing the code block function using LLM is about 103 hours. The summarized results are stored locally, so they are retrieved from the local storage during retrieval. The time for joint

Table 6 ID matching performance of different segmentation strategies at varies truncation sizes

Model StarCoderBase-1B	ID matching									
Truncation size	Segmentation strategy	Python		Java		Typescript		C#		AVG	
		EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
512	Fixed-size	12.76	44.57	11.78	46.51	**7.78**	**36.98**	10.86	37.18	10.8	41.31
	Adaptative	**13.4**	**46.01**	**12.44**	**47.13**	7.48	34.82	**11.99**	**37.94**	**11.33**	**41.48**
1024	Fixed-size	16.25	47.34	13.79	47.36	8.79	**37.2**	13.63	38.77	13.12	42.67
	Adaptative	**18.05**	**49.22**	**16.22**	**49.21**	**9.42**	36.51	**15.44**	**40.72**	**14.78**	**43.92**
2048	Fixed-size	21.5	51.82	17.16	49.42	11.35	**38.68**	17.36	42.69	16.84	45.65
	Adaptative	**23.49**	**53.74**	**19.73**	**51.22**	**12.1**	37.63	**19.63**	**44.75**	**18.74**	**46.84**
4096	Fixed-size	25.48	55.21	21.37	51.83	13.23	**39.67**	20.59	46.66	20.17	48.34
	Adaptative	**27.58**	**57.02**	**24.31**	**54.08**	**14.87**	38.62	**22.4**	**47.88**	**22.29**	**49.4**
7500	Fixed-size	29.19	58.29	24.73	52.63	12.43	**36.25**	22.85	47.81	22.3	**48.75**
	Adaptative	**30.32**	**59.4**	**24.87**	**52.63**	**12.63**	34.42	**23.47**	**48.18**	**22.82**	48.66

retrieval using BM25 and UniXcoder is 722.71 seconds. When the cross-file context truncation size is 512, 1024, 2048, 4096, and 7500, the DeepSeek-1.3B model performs inference in 3.51 minutes, 4.54 minutes, 7.11 minutes, 11.58 minutes, and 19 minutes respectively. When the truncation size is 7500, the time for DeepSeek-6.7B to perform inference is 73 minutes. We find that different inference sizes have a certain impact on the efficiency of the method. The size of the model will also affect the efficiency of our method. The most inefficient part of our method is using the LLM for code summarization. The more code blocks there are, the greater the time overhead.

Overall, our approach outperforms the current state-of-the-art methods on almost all LLMs.

6.2 Performance of Each Component

We evaluate the effectiveness of each component on the benchmark described in Section 5, including StarCoderBase, StarCoder2, and DeepSeek-Coder. We evaluate the performance of various models employing both a textual similarity-only retrieval, functional similarity-only retrieval and a combined textual and functional similarity retrieval setting with a truncation size of 4096. In our study, the former setting is the baseline. This truncation size allows for the fusion of as much information as possible with minimal information loss (Liang et al. 2024) and therefore allows for an effective comparison of the impact of different retrieval methods on the results. Tables 3 and 4 display the performance results with code matching and ID matching as the evaluation criteria. The w/o FSR denotes that only textual similarity retrieval is used and no functional similarity retrieval is used. The w/o TSR denotes that only functional similarity retrieval is used and no textual similarity retrieval is used. "Our" refers to our proposed approach, which combines both textual similarity and functional similarity retrieval. This approach significantly outperforms the baselines that rely solely on either textual similarity or functional similarity retrieval across different LLMs.

The implementation of a dual similarity retrieval method has demonstrated enhanced performance across various models. This improvement is observed across four program-

ming languages: Python, Java, TypeScript, and C#. As can be seen in Tables 3 and 4, the overall performance of the model decreases after removing any of the retrieval methods, indicating that each retrieval method contributes to the effectiveness of our method. The performance degradation is more pronounced when only text retrieval methods are used. In terms of the Code Matching metric, using different models improves the performance of EM more than ES. For example, when using DeepSeekCoder-7B, our approach improves by 18.1% over textual-only retrieval and 2.2% over functional-only retrieval on the EM metric. While on the ES metric, our approach improves 3.4% over textual-only retrieval and 1.6% over functional-only retrieval. The utilization of the dual similarity retrieval technique results in enhanced performance metrics for ID matching across multiple models in four programming languages. Using different models also improves the performance of EM more than F1. When using DeepSeekCoder-7B, our approach improves 16.3% over textual-only retrieval and 2.6% over functional-only retrieval on the EM metric. While on the F1 metric, our approach improves 6.4% over textual-only retrieval and 2.7% over functional-only retrieval. EM requires more exact matching, and the model significantly improves the accuracy of the generated code by combining textual retrieval and functional retrieval. As can be seen from Table 4, the model excels at capturing the core logic in the code, so the EM metrics are also significantly improved.

In summary, these improvements suggest that textual and functional retrieval methods are complementary; their combination leads to better model performance.

6.3 Contribution of Adaptive Segmentation Strategy

We evaluate the performance scalability of the fixed cross-file code chunk size and adaptive segmentation strategy for different truncation sizes (from 512 to 4096 tokens). From the results in Tables 3 and 4, we use the performance of StarCoderBase-1B as a baseline. Therefore, we use StarCoderBase-1B to evaluate the performance of different strategies in this section. In this experiment, the query is used for retrieval without performing any truncation. The results are shown in Tables 5 and 6, where the performance of both the fixed cross-file code chunks size and the adaptive segmentation strategy improves as the truncation size increases. The performance of the adaptive segmentation strategy for Python, Java, and C# languages consistently outperforms the performance of the fixed cross-file code chunks size on both sub-metrics of code matching. In particular, the EM scores on the three languages improve up to 2.21, 2.71, and 2.32, respectively. The ES scores for python and java improve up to 1.28 and 1.18, respectively. The ES score for C# shows smaller improvement. Typescript's ES score shows poor performance at a truncation size of 512, and although the gap decreases as the truncation size increases, there is still a performance degradation.

On both sub-metrics of ID matching, the performance of the adaptive segmentation strategy for the Python, Java, and C# languages similarly consistently outperforms the performance for fixed cross-file code chunks size. In particular, the EM score on the three languages improves by 2.1, 2.94, and 2.27, respectively. The F1 score on the three languages improves by 1.92, 2.25, and 2.06, respectively. The EM score for Typescript shows negative values at a truncation size of 512, but gradually improves by 1.64 as the truncation size increases. F1 score shows similar results to the ES metric, that is, it shows a decrease in performance. In terms of overall average performance, the adaptive segmentation strategy has a greater improvement effect on the ID matching metric than on the code matching met-

ric. When the truncation size increases from 4096 to 7500, it can be observed that the EM metric improves overall for both strategies, with the fixed-size strategy showing a greater increase. However, even with this improvement, its performance remains inferior to that of the adaptive strategy. The adaptive strategy experiences a slight decrease in both ES and F1 metrics, while the fixed-size strategy shows a decline in ES but a small improvement in F1. In terms of stability and balance, 4096 appears to be a more optimal choice. In summary, we consider that the proposed adaptive segmentation strategy performs favorably.

We further evaluate the performance of other models using a fixed-size segmentation strategy on Typescript with the truncation size of 4096, as shown in Table 7. The performance of these models using an adaptive segmentation strategy on Typescript is detailed in Tables 3 and 4. Upon comparison, we observe that all metrics decrease when using StarCoderBase-3B, StarCoderBase-7B, and StarCoder2-3B. With StarCoder2-7B, all metrics also decrease except for the EM metric under Code Match, which increases. When using StarCoderBase-1B, two metrics improve while two others decrease. However, all metrics improve with DeepSeek's model. The comprehensive performance results for all models on Typescript show fluctuations. We further verify whether the segmentation strategy influences the performance improvement effect of the model on Python language. We compared Table 7 with Tables 3 and 4. In Python, all metrics increase when using the adaptive segmentation strategy.

We observe that, compared to fixed-size segmentation, adaptive segmentation improves the EM scores for TypeScript, but at the expense of lower ES and F1 scores. This is because adaptive segmentation better preserves complete code structures such as interfaces and functions, making it easier for the model to understand variable types and code logic. As a result, when the model correctly interprets the context, it is more likely to generate outputs that are exact matches to the reference, thus boosting the EM scores. If the model makes an initial error in interpreting a type or structure, it tends to continue generating code based on this mistake, producing outputs that appear plausible but are semantically incorrect. These substantial deviations from the ground truth lead to lower ES and F1 scores. For TypeScript, adaptive segmentation makes the model more accurate when correct, but causes more severe failures when wrong.

We evaluated the performance of function-level segmentation using the DeepSeekCoder-7B model, with a truncation size of 4096. Function-level segmentation refers to segmenting the source code into independent top-level syntactic units, such as classes, functions, methods, and interfaces, where each chunk contains a complete function body without internal subdivision. As shown in Table 8, our proposed adaptive code segmenting strategy outperforms function-level segmenting across all metrics. We consider that the limitation of function-level segmenting lies in its coarse granularity, as it primarily captures the overall functionality of a function. In statement-level code completion tasks, such coarse partitioning tends to dilute critical local contextual information. In contrast, our fine-grained and

Table 7 Performance of fixed-size segmentation strategies over all LLMs

Model	Python		Python		Typescript		Typescript	
	EM	ES	EM	F1	EM	ES	EM	F1
StarCoderBase-3B	20.83	67.96	30.66	59.78	11.14	51.04	16.66	43.18
StarCoderBase-7B	23.68	70.11	33.7	62.21	12.81	50.74	18.18	43.73
StarCoder2-3B	23.3	69.79	32.98	61.84	12.54	49.08	17.97	43.04
StarCoder2-7B	24.02	70.85	33.96	62.74	12.87	52.48	18.86	45.79
DeepSeek-1B	21.31	68.05	30.81	59.4	14.69	61.78	21.31	53.91
DeepSeek-7B	26.98	72.81	37.45	65.35	20.41	65.96	27.03	59.66

Table 8 Comparison of function-level and adaptive segmentation strategies

DeepSeek-7B	Strategy	Python		Java		Typescript		C#		Avg	
Code matching		EM	ES	EM	ES	EM	ES	EM	ES	EM	ES
	Function-level	24.94	71.32	22.16	66.12	20.1	65.97	23.48	65.77	22.67	67.3
	our	**28.37**	**73.24**	**24.92**	**66.43**	**22.11**	**66.95**	**24.49**	**65.91**	**24.97**	**68.13**
ID matching		EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
	Function-level	34.66	63.27	30.72	59.05	26.98	59.39	27.69	51.01	30.01	58.18
	our	**38.72**	**66.23**	**34.22**	**60.30**	**29.11**	**60.76**	**29.3**	**51.89**	**32.84**	**59.8**

structure-aware adaptive segmenting strategy more effectively preserves the local structural and semantic context relevant to the current statement.

We compared the performance of RLCoder's split-aggregate candidate code snippet construction strategy, and our adaptive segmentation strategy when using the same retrieval methods and the same backbone model. The retrieval methods used were BM25 and UniXCoder, and the backbone model was DeepSeek-7B. When using DeepSeek-7B, the context settings followed the description in Section 6.1. Our results only reflect textual retrieval performance and do not include functional retrieval. The comparison results are shown in Table 9. From the table, we observe that our adaptive segmentation strategy outperforms the split-aggregate candidate code snippet construction strategy on most metrics, especially under the BM25 retrieval model. For RLCoder's method, UniXCoder achieves significantly better performance than BM25, particularly in terms of the EM metric. However, for our adaptive segmentation strategy, BM25 performs comparably to UniXCoder, and in some cases even slightly better. Although applying SFT (Supervised Fine-Tuning) to UniXCoder can improve RLCoder's performance to some extent, it still falls short of our approach in terms of EM. The experimental results demonstrate the effectiveness and superiority of our proposed adaptive segmentation strategy for the current task.

Overall, our adaptive strategy proves effective, but in Typescript, the choice of model significantly impacts performance results.

6.4 Affection of Semantic Model

We evaluated the impact of different algorithms on the functional retrieval performance results when using StarCoderBase-1B, as shown in Table 10. We compared the BM25, GraphCodeBert, and UniXcoder models. Different rules are followed to compute similarity scores using these three algorithms, where the BM25 algorithm computes similarity scores based on the degree of keyword matching between the functional summarizations of the

Table 9 Code matching performances of different database construction methods

Retrieval model	Method	Python		Java	
		EM	ES	EM	ES
BM25	RLCoder	18.31	68.38	17.48	65.23
	Our	**28.37**	**73.24**	**24.92**	**66.43**
UniXCoder	RLCoder	23.30	70.84	22.49	**66.78**
	RLCoder-UniXCoder-SFT	27.28	72.90	25.11	66.39
	Our	**28.22**	**73.16**	**25.20**	66.16

Table 10 Performances of semantic representation for functional retrieval

Model	Code matching									
	Python		Java		Typescript		C#		AVG	
	EM	ES	EM	ES	EM	ES	EM	ES	EM	ES
GraphCodeBert	11.56	62.29	12.58	61.04	5.66	43.11	13.07	63.61	10.72	57.51
BM25	18.84	66.21	18.42	63.3	7.57	44.48	16.23	65.5	15.27	59.87
UniXcoder	**21.99**	**68.17**	**19.4**	**63.88**	**8.37**	**45.12**	**19.17**	**66.65**	**17.23**	**60.96**
Model	ID matching									
	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
GraphCodeBert	20.08	51.75	20.66	51.84	10.19	34.54	17.14	43.17	17.02	45.33
BM25	28.56	57.89	26.65	55.42	12.63	36.75	20.25	45.95	22.02	49
UniXcoder	**32.05**	**60.69**	**27.54**	**56.46**	**13.44**	**37.22**	**23.47**	**48.01**	**24.13**	**50.6**

query and the retrieved code blocks. In contrast, when performing functional retrieval using the GraphCodeBert and UniXcoder models, the semantic vector representations of functional summaries for the query and the retrieved code blocks are obtained from the models, and the cosine similarity between the two vectors is then calculated. As can be seen from Table 10, GraphCodeBert shows the worst performance results, while UniXcoder has the best performance results. Combining the average performance of the four languages, the values of the four metrics are improved by 6.51, 3.45, 7.11, and 5.27, respectively. The performance of UniXcoder is improved by 1.96, 1.09, 2.11, and 1.6, respectively, concerning the performance of BM25. The results presented in Tables 3 and 4 are obtained by performing functional retrieval using UniXcoder. It can be seen that when combined with textual retrieval, the method proposed in this study improves the four metrics by 0.97, 1.14, 1.11, and 1.21, respectively, relative to using only functional retrieval.

To summarize, this comparison shows that the choice of retrieval algorithm affects the performance of our method. UniXcoder model contributes to the performance of the proposed method.

6.5 Case Study

We validated the effectiveness of our approach using the DeepSeek-7B model on the executable benchmark DyPyBench (Bouzenia et al. 2024), truncating the cross-file context to 512 tokens. Spiess et al. (2025) constructed a test set based on DyPyBench to assess in-file code completion performance, but it did not consider cross-file context. Therefore, we constructed a test set tailored for repository-level code completion tasks based on this benchmark. DyPyBench contains 50 code repositories. From each repository, we randomly selected Python files, excluding those in the ”tests” directory, and then randomly selected two functions with test coverage between 85% and 100% from these files. Within each selected function, we chose a valid line of code from the function body as the line to be completed (excluding blank lines and comment lines). Our approach achieves a pass rate of 38.23% on this test set. Table 11 displays examples of both successful and failed completions by our approach. We manually checked these examples to analyze the reasons for the success and failure of the model. For successful examples, the code follows clear naming conventions and structural patterns, the context of the code to be completed is rich in type hint information, and the retrieved cross-file code has structural integrity. For failed examples, many functions have very long implementation logic and string parsing requirements,

Table 11 Performance on executable benchmark

Results	File	Ground-truth code	lineno
Success cases	moviepy-master/moviepy/ video/fx/Scroll.py	y = int(max(0, min(y_max, self.y_start + round(self.y_speed * t))))	53
	Supervisor-main/supervisor/ supervisorctl.py	sys.exit (c.exitstatus)	1416
	thefuck/rules/ apt_invalid_operation.py	return replace_command(command, invalid_operation, operations)	62
	Blinker-main/src/blinker/base.py	if iscoroutinefunction (receiver)	242
Failed cases	kshare-main/akshare/futures/futures_hq_sina.py	item.strip().split("=") [1] .split(",")	139
	main/akshare/futures/futures_hq_sina.py	here = os.path.abspath (dirname(__file__))	15
	main/src/click/ termui.py	click.echo ("Invalid input")	166
	pydub-master/pydub/util.py	file, close_file = _fd_or_path_or_tempfile (filepath, 'rb', tempfile=False)	270

requiring the model to maintain context understanding in long sequences, for example, some contexts are as long as 983 lines. Due to the LLM's context window limit, these long contexts were truncated during input. Other code snippets interacts with temporary files or network requests, requiring the model to understand side effects, which makes it challenging for the model to make predictions.

7 Threats to Validity

Although our method achieves state-of-the-art performance, we are aware of some factors that may affect its effectiveness. Furthermore, we discuss potential directions for optimizing our approach.

Expanding the Language Range CrossCodeEval includes four programming languages, so we consider the segmentation structure of each language separately when building the retrieval database. This limits the generality of the method to a certain extent, especially when dealing with multiple programming languages. Therefore, it is necessary to explore more universal segmentation strategies that go beyond the fixed-size code block segmentation and are not dependent on language characteristics.

Real-world Data We demonstrate the effectiveness of the method on the popular repository-level code completion datasets. Although the test was conducted on the real-world open-source GitHub project DyPyBench, the test data is relatively small, and further verification on larger-scale of real code bases or other executable benchmarks is still needed to evaluate the applicability and robustness of the method in actual development scenarios.

Efficiency vs. Time Cost Although using an LLM to summarize cross-file code blocks allows for local storage, it introduces latency when generating code summaries. In addition, the quality of the summary generated by the LLM directly affects the retrieval performance. When implementing functional similarity retrieval, we adopted a semantic vector extraction method with better performance, which also increased the retrieval delay.

Potential Optimization The code function summary based on the LLM in our method brings a large time overhead. In the future, we can explore the use of static analysis techniques (such as AST-based key node extraction, lexical key token selection or the extraction of control flow and data flow graphs) to build a functional representation of the code block, thereby reducing reliance on large models and avoiding this part of the time overhead. In addition, our method performs suboptimally on function generation tasks such as those evaluated in Â3 CodGen (Liao et al. 2024). This is likely mainly due to the simplicity of our prompt design, which focuses on retrieving and integrating relevant cross-file context rather than providing structured, rule-based instructions. As a result, our approach is more suitable for single-line or local code completion tasks, rather than complex, structured function generation. Future work will aim to improve the prompt structure to enhance performance on complex code generation tasks. Our evaluation reveals that the proposed method does not achieve optimal performance on certain metrics for Java and Typescript. In Java, our approach exhibits limitations in adapting to global class structures. In TypeScript, however, it demonstrates a stronger ability to accurately parse local semantics. Future work should focus on building a language-aware dynamic context organization mechanism to overcome the limitations of static segmenting strategies, enabling flexible aggregation of local information based on language-specific characteristics, while using semantic analysis to recover long-range dependencies, thus balancing local precision with global consistency.

8 Conclusion

In this study, we used an adaptive segmentation strategy and a fused retrieval method to address repository-level code completion problems. We propose a framework comprising two components: an adaptive segmentation strategy and an LLM-augmented retrieval module. The adaptive segmentation strategy splits all source code into code chunks of different sizes with functional logic to build the retrieval database. The LLM-augment retrieval module uses DeepSeek-v2 to functionally summarize the code chunks, based on which functional similarity retrieval is implemented, combined with textual similarity retrieval to get a more effective cross-file context. Equipped with powerful LLMs, the inference module generates the code completion based on both the current file and cross-file contexts. Comparing various LLMs, our method achieves better performance and improvement on the benchmark. Our findings demonstrate the competitiveness for repository-level code completion tasks with RAG-based approaches, and the potential to enhance the quality and robustness of cross-file information retrieval. Future research on repository-level code completion can be enhanced with more advanced LLM-enhanced retrieval approaches.

Author Contributions Conceptualization, Methodology, Software, Validation, Writing-original draft: Yuanyuan Shen; Writing-review and editing: Pinle Qin, Kaiyi Zhao, Fuwei Zhang; Data collection: Fan Zhang, Guiji Li. All the authors approve the final articles.

Funding This work was supported by the Youth Project of Fundamental Research Program (Free Exploration Category) of Shanxi Province (Grant Nos. 202403021212168, 202303021222098), Natural Science Foundation of Hunan Province (Grant Nos. 2022JJ40527), and Scientific Research Foundation of Hunan Provincial Education Department (Grant Nos. 21B0760).

Data availability statement Data and code for this manuscript are available at https://github.com/fioushen/repository-level-code-completion/tree/master.

Declarations

Ethical Approval Not applicable.

Informed Consent Not applicable.

Conflict of Interest No conflict of interest exists in the submission of this manuscript, and manuscript is approved by all authors for publication. The work described was original research that has not been published previously, and not under consideration for publication elsewhere, in whole or in part.

Clinical Trial Number Not applicable.

References

Allal LB, Li R, Kocetkov D, Mou C, Akiki C, Ferrandis CM, Muennighoff N, Mishra M, Gu A, Dey M et al (2023) SantaCoder: don't reach for the stars!. https://doi.org/10.48550/arXiv.2301.03988

Athiwaratkun B, Gouda SK, Wang Z, Li X, Tian Y, Tan M, Ahmad WU, Wang S, Sun Q, Shang M et al (2023) Multi-lingual evaluation of code generation models. https://doi.org/10.48550/arXiv.2210.14868

Biderman S, Schoelkopf H, Anthony QG, Bradley H, O'Brien K, Hallahan E, Khan MA, Purohit S, Prashanth US, Raff E et al (2023) Pythia: a suite for analyzing large language models across training and scaling. In: International Conference on Machine Learning (ICML 2023), pp 2397–2430. https://proceedings.mlr.press/v202/biderman23a.html

B Y, N Z, SP L, X X (2020) Survey of intelligent code completion. J Softw (JOS) 31(5):1435–1453. https://doi.org/10.13328/j.cnki.jos.005966

Bouzenia I, Pradel M (2025) You name it, i run it: an llm agent to execute tests of arbitrary projects. Proc ACM Softw Eng 2(ISSTA):1054–1076. https://doi.org/10.1145/3728922

Bouzenia I, Krishan BP, Pradel M (2024) Dypybench: a benchmark of executable python software. ACM Int Conf Foundations Softw Eng (FSE) 1(FSE). https://doi.org/10.1145/3643742

Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A et al (2020) Language models are few-shot learners. Adv Neural Inf Process Syst (NeurIPS 2020) 33:1877–1901. https://doi.org/10.5555/3495724.3495883

Carlini N, Tramer F, Wallace E, Jagielski M, Herbert-Voss A, Lee K, Roberts A, Brown T, Song D, Erlingsson U et al (2021) Extracting training data from large language models. In: 30th USENIX Security Symposium (USENIX Security 21), pp 2633–2650. https://www.usenix.org/conference/usenixsecurity21/presentation/carlini-extracting

Chen J, Pan Y, Li Y, Yao T, Chao H, Mei T (2023) Retrieval augmented convolutional encoder-decoder networks for video captioning. ACM Trans Multimed Comput Commun Appl (TOMCCAP) 19(1s):1–24. https://doi.org/10.1145/3539225

Chen J, Hu X, Li Z, Gao C, Xia X, Lo D (2024a) Code search is all you need? improving code suggestions with code search. In: 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE), pp 880–892. https://doi.org/10.1145/3597503.3639085

Chen M, Tian H, Liu Z, Ren X, Sun J (2024b) JumpCoder: go beyond autoregressive coder via online modification. In: Proceedings of the 62nd annual meeting of the association for computational linguistics (ACL, Volume 1: Long Papers), pp 11500–11520. https://doi.org/10.18653/v1/2024.acl-long.619

Chen M, Tworek J, Jun H, Yuan Q, Pinto HPdO, Kaplan J, Edwards H, Burda Y, Joseph N, Brockman G et al (2021) Evaluating large language models trained on code. arXiv:2107.03374. https://doi.org/10.48550/arXiv.2107.03374

Chowdhery A, Narang S, Devlin J, Bosma M, Mishra G, Roberts A, Barham P, Chung HW, Sutton C, Gehrmann S et al (2023) Palm: scaling language modeling with pathways. J Mach Learn Res 24(240):1–113. https://doi.org/10.5555/3648699.3648939

Christopoulou F, Lampouras G, Gritta M, Zhang G, Guo Y, Li Z, Zhang Q, Xiao M, Shen B, Li L et al (2022) Pangu-coder: program synthesis with function-level language modeling. arXiv:2207.11280. https://doi.org/10.48550/arXiv.2207.11280

Di P, Li J, Yu H, Jiang W, Cai W, Cao Y, Chen C, Chen D, Chen H, Chen L et al (2024) Codefuse-13b: a pretrained multi-lingual code large language model. In: Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP 2024), pp 418–429. https://doi.org/10.1145/3639477.3639719

Ding Y, Wang Z, Ahmad WU, Ding H, Tan M, Jain N, Ramanathan MK, Nallapati R, Bhatia P, Roth D, Xiang B (2023) Crosscodeeval: a diverse and multilingual benchmark for cross-file code completion. In: Thirty-seventh conference on neural information processing systems datasets and benchmarks track (NeurIPS 2023). https://openreview.net/forum?id=wgDcbBMSfh

Ding Y, Wang Z, Ahmad WU, Ramanathan MK, Nallapati R, Bhatia P, Roth D, Xiang B (2021) Leveraging passage retrieval with generative models for open domain question answering. In: Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2021), pp 874–880. https://doi.org/10.18653/v1/2021.eacl-main.74

Ding Y, Wang Z, Ahmad WU, Ramanathan MK, Nallapati R, Bhatia P, Roth D, Xiang B (2024) Cocomic: code completion by jointly modeling in-file and cross-file context. In: Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024), pp 3433–3445. https://aclanthology.org/2024.lrec-main.305/

Eghbali A, Pradel M (2024) De-Hallucinator: mitigating LLM hallucinations in code generation tasks via iterative grounding. https://doi.org/10.48550/arXiv.2401.01701

Guo D, Lu S, Duan N, Wang Y, Zhou M, Yin J (2022) Unixcoder: unified cross-modal pre-training for code representation. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL 2022), pp 7212–7225. https://doi.org/10.18653/v1/2022.acl-long.499

Guo D, Zhu Q, Yang D, Xie Z, Dong K, Zhang W, Chen G, Bi X, Wu Y, Li Y et al (2024) Deepseek-coder: when the large language model meets programming–the rise of code intelligence. arXiv:2401.14196. https://doi.org/10.48550/arXiv.2401.14196

Hendrycks D, Basart S, Kadavath S, Mazeika M, Arora A, Guo E, Burns C, Puranik S, He H, Song D, Steinhardt J (2021) Measuring coding challenge competence with APPS. https://doi.org/10.48550/arXiv.2105.09938

Huang Y, Huang J (2024) A survey on retrieval-augmented text generation for large language models. https://doi.org/10.48550/arXiv.2404.10981

Huang D, Zhang JM, Luck M, Bu Q, Qing Y, Cui H (2024) AgentCoder: multi-agent-based code generation with iterative testing and optimisation. https://doi.org/10.48550/arXiv.2312.13010

Izadi M, Gismondi R, Gousios G (2022) Codefill: multi-token code completion by jointly learning from structure and naming sequences. In: Proceedings of the 44th International Conference on Software Engineering (ICSE 2022), pp 401–412. https://doi.org/10.1145/3510003.3510172

Izadi M, Katzy J, Van Dam T, Otten M, Popescu RM, Van Deursen A (2024) Language models for code completion: a practical evaluation. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE 2024), pp 1–13. https://doi.org/10.1145/3597503.3639138

Jin M, Shahriar S, Tufano M, Shi X, Lu S, Sundaresan N, Svyatkovskiy A (2023) Inferfix: end-to-end program repair with llms. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2023), pp 1646–1656. https://doi.org/10.1145/3611643.3613892

Kim J, Nam J, Mo S, Park J, Lee S-W, Seo M, Ha J-W, Shin J (2024) Sure: summarizing retrievals using answer candidates for open-domain QA of LLMs. In: The Twelfth International Conference on Learning Representations (ICLR 2024). https://iclr.cc/media/iclr-2024/Slides/17509.pdf

Kukich K (1992) Techniques for automatically correcting words in text. ACM Comput Surv (CSUR) 24(4):377–439. https://doi.org/10.1145/146370.146380

Li JA, Li Y, Li G, Hu X, Xia X, Jin Z (2021) Editsum: a retrieve-and-edit framework for source code summarization. In: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE 2021). IEEE, pp 155–166. https://doi.org/10.1109/ASE51524.2021.9678724

Li R, Allal LB, Zi Y, Muennighoff N, Kocetkov D, Mou C, Marone M, Akiki C, Li J, Chim J et al (2023) StarCoder: may the source be with you!. https://doi.org/10.48550/arXiv.2305.06161

Liang M, Xie X, Zhang G, Zheng X, Di P, Jiang, Chen H, Wang C, Fan G (2024) REPOFUSE: repository-level code completion with fused dual context. https://doi.org/10.48550/arXiv.2402.14323

Liao D, Pan S, Sun X, Ren X, Huang Q, Xing Z, Jin H, Li Q (2024) A3-codgen: a repository-level code generation framework for code reuse with local-aware, global-aware, and third-party-library-aware. IEEE Trans Softw Eng (TSE) (01):1–16. https://doi.org/10.1109/TSE.2024.3486195

Liu Z, Chen C, Wang J, Chen M, Wu B, Tian Z, Huang Y, Hu J, Wang Q (2024a) Testing the limits: unusual text inputs generation for mobile app crash detection with large language model. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE 2024), pp 1–12. https://doi.org/10.1145/3597503.3639118

Liu T, Xu C, McAuley J (2023) RepoBench: benchmarking repository-level code auto-completion systems. https://doi.org/10.48550/arXiv.2306.03091

Liu W, Yu A, Zan D, Shen B, Zhang W, Zhao H, Jin Z, Wang Q (2024b) Graphcoder: enhancing repository-level code completion via code context graph-based retrieval and language model. arXiv:2406.07003. https://doi.org/10.48550/arXiv.2406.07003

Lozhkov A, Li R, Allal LB, Cassano F, Lamy-Poirier J, Tazi N, Tang A, Pykhtar D, Liu J, Wei Y et al (2024) StarCoder 2 and The Stack v2: the next generation. https://doi.org/10.48550/arXiv.2402.19173

Mallen A, Asai A, Zhong V, Das R, Khashabi D, Hajishirzi H (2023) When not to trust language models: investigating effectiveness of parametric and non-parametric memories. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023). https://doi.org/10.18653/v1/2023.acl-long.546

Nijkamp E, Hayashi H, Xiong C, Savarese S, Zhou Y (2023a) Codegen2: lessons for training llms on programming and natural languages. arXiv:2305.02309. https://doi.org/10.48550/arXiv.2305.02309

Nijkamp E, Pang B, Hayashi H, Tu L, Wang H, Zhou Y, Savarese S, Xiong C (2023b) CodeGen: an open large language model for code with multi-turn program synthesis. https://doi.org/10.48550/arXiv.2203.13474

OpenAI (2023) Introducing chatgpt. https://openai.com/blog/chatgpt

Parvez MR, Ahmad W, Chakraborty S, Ray B, Chang K-W (2021) Retrieval augmented code generation and summarization. In: Findings of the Association for Computational Linguistics (EMNLP 2021), pp 2719–2734. https://doi.org/10.18653/v1/2021.findings-emnlp.232

Phan HN, Phan HN, Nguyen TN, Bui ND (2024) Repohyper: better context retrieval is all you need for repository-level code completion. arXiv:2403.06095. https://doi.org/10.48550/arXiv.2403.06095

Ramos R, Martins B, Elliott D, Kementchedjhieva Y (2023) Smallcap: lightweight image captioning prompted with retrieval augmentation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2023), pp 2840–2849. https://doi.org/10.1109/CVPR52729.2023.00278

Robertson S, Zaragoza H et al (2009) The probabilistic relevance framework: Bm25 and beyond. Found Trends® Inf Retriev 3(4):333–389. https://doi.org/10.1561/1500000019

Roziere B, Gehring J, Gloeckle F, Sootla S, Gat I, Tan XE, Adi Y, Liu J, Remez T, Rapin J et al (2023) Code llama: open foundation models for code. arXiv:2308.12950. https://doi.org/10.48550/arXiv.2308.12950

Shrivastava D, Kocetkov D, Vries H, Bahdanau D, Scholak T (2023) RepoFusion: training code models to understand your repository. https://doi.org/10.48550/arXiv.2306.10998

Spiess C, Gros D, Pai KS, Pradel M, Rabin MRI, Alipour A, Jha S, Devanbu P, Ahmed T (2025) Calibration and correctness of language models for code. In: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), pp 540–552. https://doi.org/10.1109/ICSE55347.2025.00040

Tsai Y, Liu M, Ren H (2024) Rtlfixer: automatically fixing rtl syntax errors with large language model. In: Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC 2024), pp 1–6. https://doi.org/10.1145/3649329.3657353

Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. Adv Neural Inf Process Syst (NIPS 2017) 30. https://doi.org/10.5555/3295222.3295349

Wang H, Xia X, Lo D, He Q, Wang X, Grundy J (2021) Context-aware retrieval-based deep commit message generation. ACM Trans Softw Eng Methodology (TOSEM) 30(4):1–30. https://doi.org/10.1145/3464689

Wang C, Zhang J, Feng Y, Li T, Sun W, Liu Y, Peng X (2025a) Teaching code llms to use autocompletion tools in repository-level code generation. ACM Trans Softw Eng Methodology (TOSEM). https://doi.org/10.1145/3714462

Wang Y, Le H, Gotmare AD, Bui ND, Li J, Hoi S (2023) Codet5+: open code large language models for code understanding and generation. In: The 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023). https://doi.org/10.18653/v1/2023.emnlp-main.68

Wang Y, Wang Y, Guo D, Chen J, Zhang R, Ma Y, Zheng Z (2025b) Rlcoder: reinforcement learning for repository-level code completion. In: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE 2025), pp 165–177. https://conf.researchr.org/details/icse-2025/icse-2025-research-track/24/RLCoder-Reinforcement-Learning-for-Repository-Level-Code-Completion

Wang Y, Wang W, Joty S, Hoi SCH (2021) CodeT5: identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP 2021). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, pp 8696–8708. https://doi.org/10.18653/v1/2021.emnlp-main.685

Wu D, Ahmad WU, Zhang D, Ramanathan MK, Ma X (2024) Repoformer: selective retrieval for repository-level code completion. arXiv:2403.10059. https://doi.org/10.48550/arXiv.2403.10059

Wu M, Cao S (2024) LLM-augmented retrieval: enhancing retrieval models through language models and doc-level embedding. https://doi.org/10.48550/arXiv.2404.05825

Xue Z, Gao Z, Wang S, Hu X, Xia X, Li S (2024) Selfpico: self-guided partial code execution with llms. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. ISSTA 2024, pp 1389–1401. https://doi.org/10.1145/3650212.3680368

Zhang F, Chen B, Zhang Y, Keung J, Liu J, Zan D, Mao Y, Lou J-G, Chen W (2023) Repocoder: repository-level code completion through iterative retrieval and generation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023), pp 2471–2484. https://doi.org/10.18653/v1/2023.emnlp-main.151

Zhao P, Zhang H, Yu Q, Wang Z, Geng Y, Fu F, Yang L, Zhang W, Cui B (2024) Retrieval-augmented generation for ai-generated content: a survey. arXiv:2402.19473. https://doi.org/10.48550/arXiv.2402.19473

Zheng Q, Xia X, Zou X, Dong Y, Wang S, Xue Y, Shen L, Wang Z, Wang A, Li Y, Su T, Yang Z, Tang J (2023) Codegeex: a pre-trained model for code generation with multilingual benchmarking on humaneval-x. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2023), pp 5673–5684. https://doi.org/10.1145/3580305.3599790

Zhu T, Li Z, Pan M, Shi C, Zhang T, Pei Y, Li X (2024) Deep is better? an empirical comparison of information retrieval and deep learning approaches to code summarization. ACM Trans Softw Eng Methodology (TOSEM) 33(3). https://doi.org/10.1145/3631975

Authors and Affiliations

Yuanyuan Shen[1] · Pinle Qin[1] · Kaiyi Zhao[1] · Fuwei Zhang[1] · Fan Zhang[2] · Guiji Li[3]

✉ Yuanyuan Shen
shenyuanyuan@nuc.edu.cn

Pinle Qin
qpl@nuc.edu.cn

Kaiyi Zhao
zhaokaiyi@nuc.edu.cn

Fuwei Zhang
20240048@nuc.edu.cn

Fan Zhang
fanzhang@hnu.edu.cn

Guiji Li
guiji.li@hnu.edu.cn

[1] School of Computer Science and Technology, North University of China, Taiyuan 030051, China

[2] College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China

[3] School of Computer Science and Engineering, Changsha University, Changsha 410022, China